POLYGLOT WORKSHOP

WORKING WITH TEXT DATASETS

PART I

Tutors
Shiza Fatimah & Nicholas Kluge

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN

AGENDA

1. Where DO we find text?

- ✓ The evolution of multilingual text corpora
- ✓ From judges to Generators
- ✓ Lessons From Cosmopedia and BeyondWeb

2. The First Filters

- ✓ Web Archiving File Formats
- ✓ URL Filtering
- ✓ Text Extraction
- ✓ Language Identification

3. Quality and Deduplication

- ✓ Quality Heuristics
- ✓ Deduplication
- ✓ Scaling Laws for Data Constrained Models

AGENDA

4. LLM-as-a-Filter and Synthetic Data Generation

✓The case for the Educational Filter

✓The crawl vs. The hub (and other sources)

5. Bonus

✓Decontamination

✓ Recipes and Mixtures

WHERE DO WE FIND TEXT?

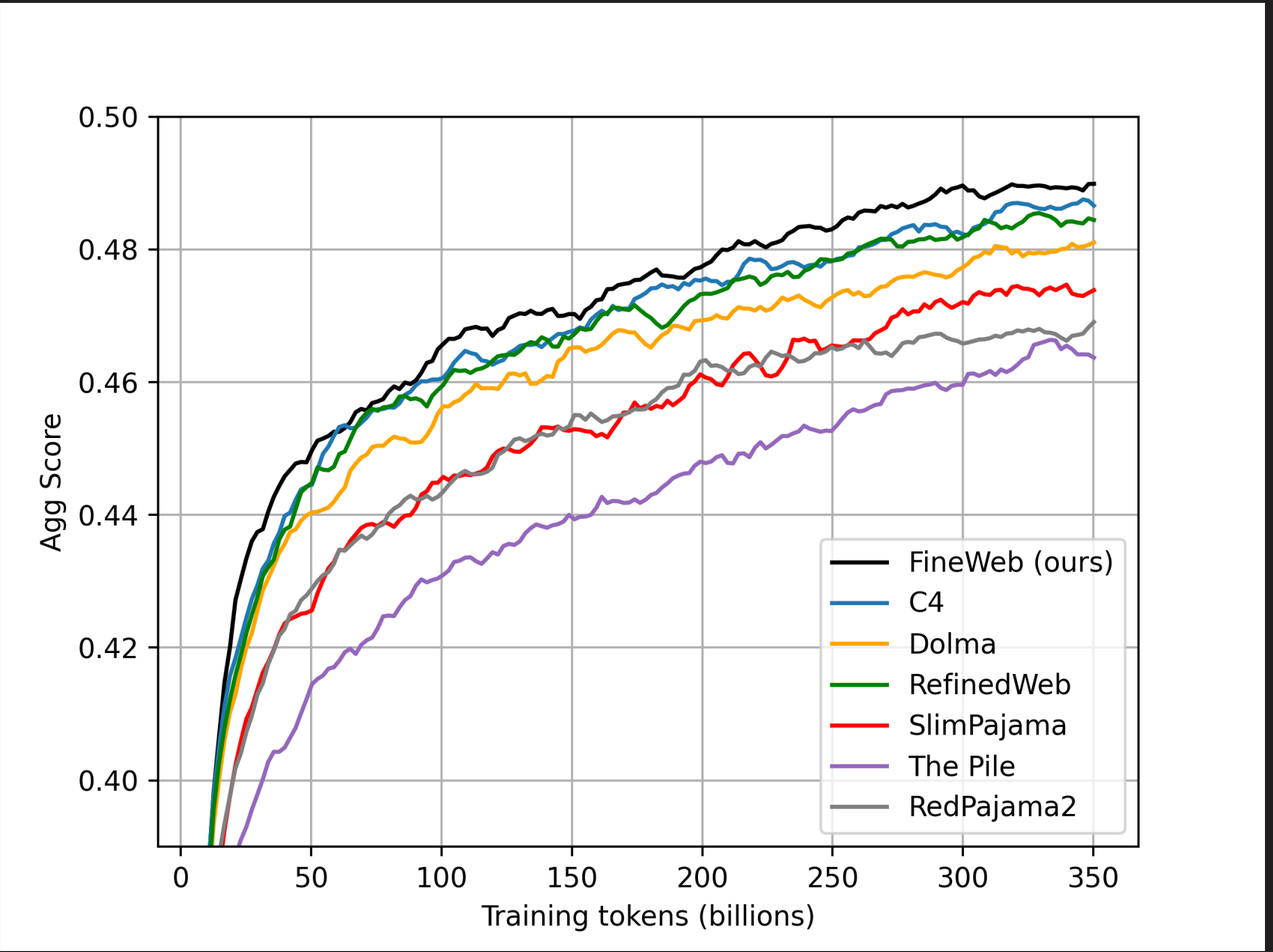
The Crawl and Public Repositories

THE EVOLUTION OF MULTILINGUAL TEXT CORPORA

"The performance of a large language model (LLM) depends heavily on the quality and size of its pretraining dataset. However, the pretraining datasets for state-of-the-art open LLMs are ***not publicly available, and very little is known about how they were created****."* Penedo et al. (2024).

- ✓ **Multilingual C4** (Raffel et al. Colossal Clean Crawled Corpus. 2019) (langdetect via CLD3)
- ✓ **OSCAR** (Suárez et al. Open Super-large Crawled. 2019) (goclassy → Ungoliant via FastText)
- ✓ **CC-100** (Calzolari et al. Common Crawl 100. 2020) (FastText + cc_net → cool trick for data quality)
- ✓ **CulturaX** (Nguyen et al. Cleaned, Enormous, and Multilingual. 2024) (mC4 → FastText + filters)
- ✓ **FineWeb 1-2** (Penedo et al. Part 1 and Part 2. 2024-2025) (+1000 languages → the one we use!)

SPOILER: WE SHOULD START FROM THE SOTA



Source → Hugging Face

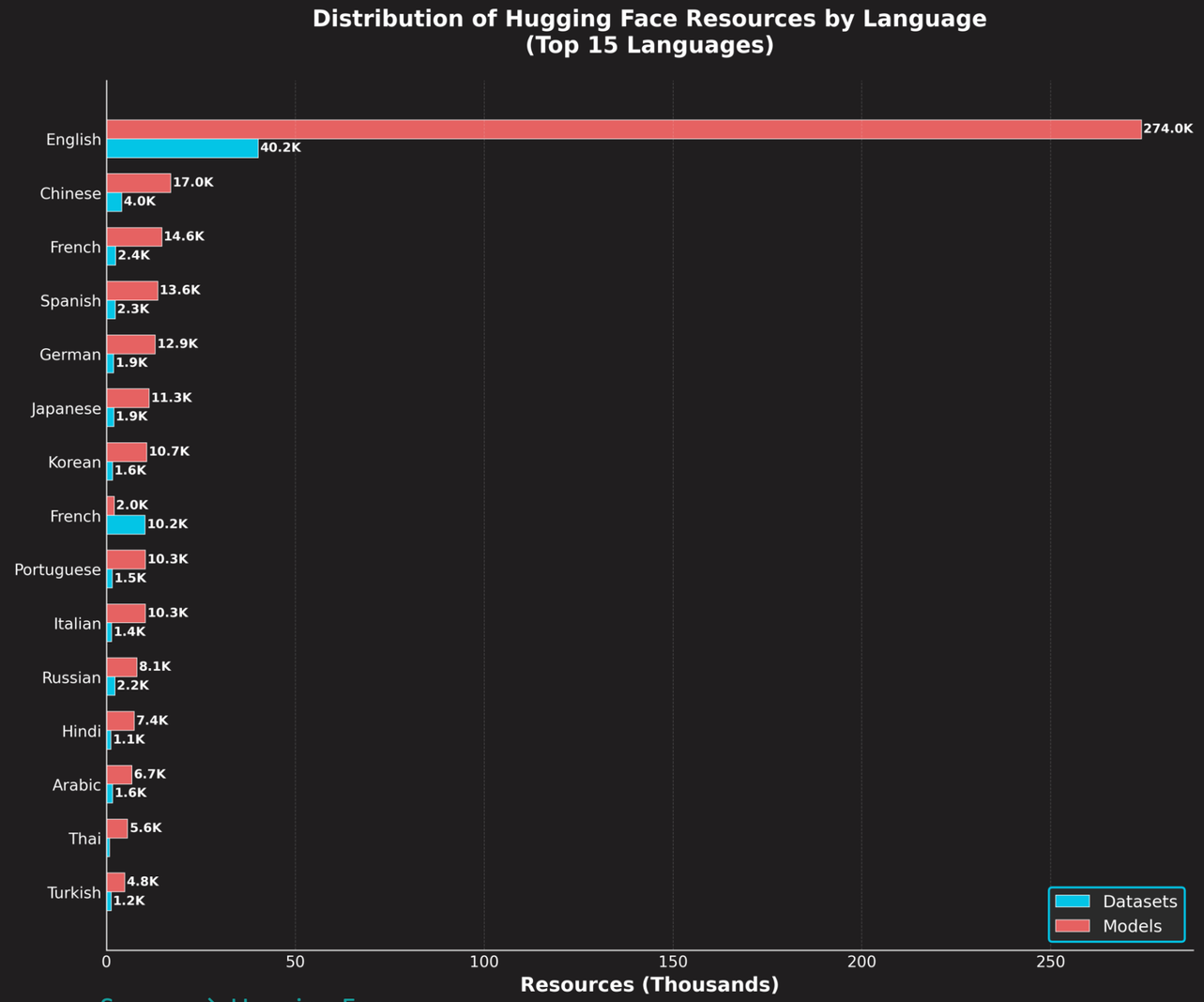
WHERE DID THE PREVIOUSLY MENTIONED WORKS GET THEIR DATA FROM?

- **The Crawl (Common Crawl):** Common Crawl is a nonprofit organization that maintains a free, open repository of web crawl data. Founded in 2007, it collects and provides access to a vast dataset comprising over 250 billion web pages and spanning 17 years of internet history. **Their focus is scale, not precision.**

- **The Hub (Hugging Face):** The Hugging Face Hub is a central platform for the machine learning community, hosting a large number of ML models and datasets. **Their focus is on accessibility**.

- *Depending on your language of interest, this might **not be enough** (but don't give up before you check!)*

WHY MAY IT NOT BE ENOUGH?



Source → Hugging Face

POLYGLOT

WHY MAY IT NOT BE ENOUGH?

- **Statistics of Common Crawl Monthly Archives:**
  - **Source**
  - **Source HF Dataset**
  - **Visualization (by Piotr Patrzyk)**

SCRAPPING AND ANALOG EXTRACTION – *NOT COVERED*

Common Crawl may be too **sparse** to be useful, and Hugging Face datasets may be **nonexistent** or **insufficient** in scale or coverage. In these cases, building a viable corpus often requires additional effort. While more labor-intensive, these approaches are sometimes the *only way* to obtain meaningful data for underrepresented languages.

- ✓ **Scraping** (e.g., Tralifatura, Scrapy), for "reasons" I cannot give you exercises or examples.
- ✓ **Analog Conversion** (e.g., we used Marker, because it is simple, fast, and does good OCR)
- ✓ **Translations** (e.g., +55 languages TranslateGemma), but your language might be so low-resource that you have no machine-translator (and human translators are expensive).

💡 **You can get away with 10k–50k parallel sentences to fine-tune an mBART. [1, 2]**

💡 With 5 translators → **100–160 hours (~2–4 weeks).**

YOU SHOULD STILL LOOK FOR YOURSELF

At the beginning of several projects, we found a few high-quality datasets for the languages we were targeting, but the absence of evidence is not evidence of absence; with sufficient digging, we uncovered relevant resources.

GigaKriya (~25 BT)

GigaLekh (~80 BT)

GigaVerbo-v2 (~400 BT)

THE CRAWL VS. THE HUB

Manual inspection and sanity checks are something we (currently) cannot avoid if we seek high quality. At some point, it is always a good practice to randomly sample some data and to spend some time *“looking at it”*. ***And this is what we are going to practice now!***

- ✔ Learn how to quickly inspect the data you are working with.
- ✔ Learn how to calculate basic statistics.
- ✔ Learn how to identify potential issues with the data.

EXERCISE

THE FIRST FILTERS

Banned Sources, Text Extraction, and Language Identification

SELECTING A FRAMEWORK / BAG-OF-TOOLS

There's absolutely nothing wrong with building on top of a solid, community-supported framework. We're not reinventing **AutoGrad**. We're not turning sand into wafers. And we're definitely not trying to build a **transformer from scratch with nothing but NumPy**, blind optimism, and a pinch of fairy dust…

We will be using **DataTrove**, a library for processing, filtering, and deduplicating text data at a very large scale. It is platform-agnostic, running out of the box locally or **on a SLURM cluster.**

- ✔ Datasets (we just used it!)
- ✔ Ray
- ✔ NVIDIA NeMo Curator
- ✔ Warcio

WEB ARCHIVING FILE FORMATS

If we are working with datasets that are already filtered, processed, and polished, we can skip some steps. But if we are working with data fresh from a Crawl, some **necessary steps are required to ensure a minimum level of quality**. While downloading some of the finest datasets ever assembled from the Hub requires only a few lines of Python, working with CC dumps is a little more involved.

Let us check what a DUMP looks like (CC-MAIN-2025-51, CC-MAIN-YEAR-WEEK):

- ✔ WARC (Web ARChive) is the industry standard for web archiving (🧶).
- ✔ WAT (Web ARChive Timestamp) is the metadata associated with the crawl (🧶).
- ✔ WET (WARC Encapsulated Text) is extracted plain text from web content (🧶).
- ✔ **What differences are evident?**

URL FILTERING

URL filtering is a **rule-based approach that blocks web pages** by matching their URLs against built-in and optional user-defined lists of banned domains, exact URLs, and undesired keywords.

```
>> URLFilter(...)
```

- ✔ Entire domains or subdomains are rejected: `example.com`, `badsite.example`
- ✔ Exact URL matches are rejected: `http://example.com/specific/page`
- ✔ If a URL contains certain words, it is filtered out: `.../pirated-movie-download`
- ✔ Partial string matches inside the URL path or parameters: `xxx`, `torrent`, `crack`

Samples of bad stuff (all safe to click): 🖇️ 🖇️ 🖇️ 🖇️

WARC → WET

WET files **contain only the body text of web pages, extracted from HTML and excluding HTML code, images, and other media.** This makes them useful for NLP tasks.

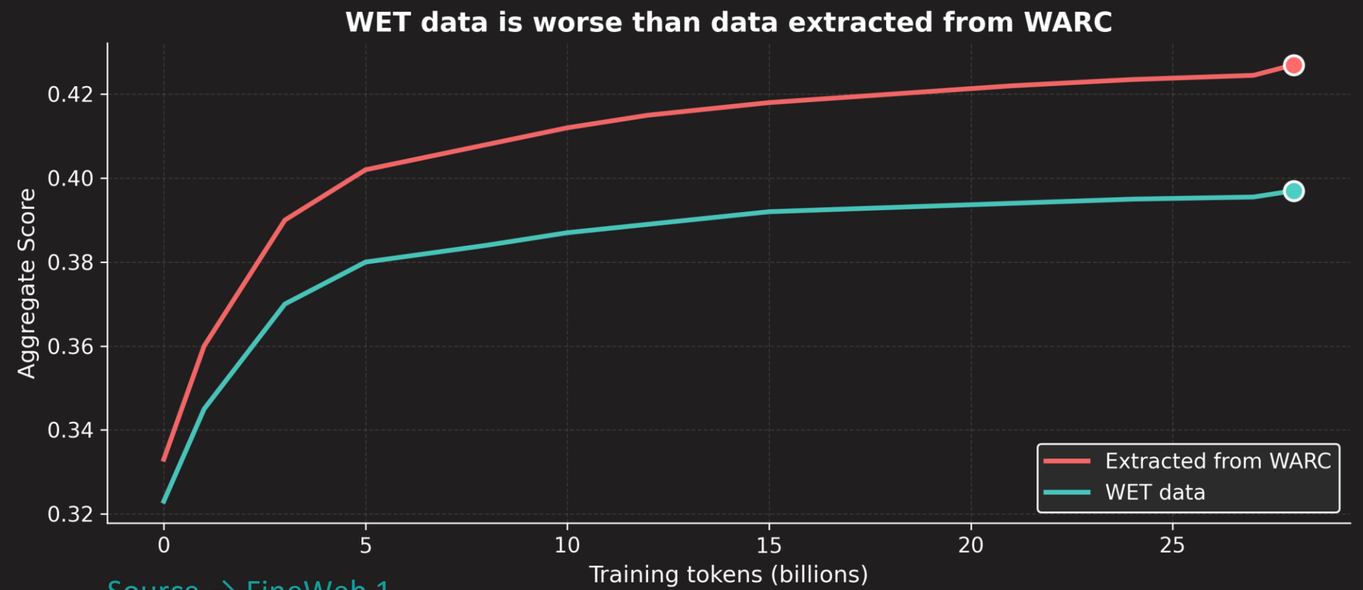
The conversion from raw WARC files into usable text (what we call ***extraction***) is a **costly process relative to later filtering steps** and serves **two primary functions**:

- ✔ Removing boilerplate HTML/XML code.
- ✔ Normalizing the text:
  - >> Unicode Normalization
  - >> Forms and Equivalence
  - >> Avoiding 🞋 🞋 🞋 🞋

HOW GOOD IS WET

Common Crawl has its own toolkit for its work (the webarchive-commons toolkit for text extraction, cld2 for language detection, etc.). However, none of this is *"great"*. It gets the job done *"in a rough"* way, but if you are aiming for **quality** over **quantity**, there are some things we can improve upon.

- ✔ **Tralifatura (🧵)**
- ✔ Removes more boilerplate (~25%)
- ✔ Normalizes (NFC) by default.
- ✔ Ablations reveal performance gains.
- ✔ WET: lower-budget option.



Source → FineWeb 1

LANGUAGE IDENTIFICATION

Language identification (**LID**) is a crucial step in any LLM development pipeline. What they do is exactly what the name suggests: ***identify the language of some piece of content.***

```
>> LanguageFilter(...)
```

- ✔ CLD2 → supports 80 languages
- ✔ CLD3 → supports +100 languages
- ✔ Langdetect → supports 55 languages → **extendable 📎 (via Java)**
- ✔ FastText → supports 176 languages → **extendable 📎 (via Python)**
- ✔ GlotLID-v3 → **+2000 labels (nicely documented) → extendable 📎 → Test it here.**

Must read: GlotLID: Language Identification for Low-Resource Languages

LANGUAGE IDENTIFICATION

All these pipeline components (i.e., **reading WARCS, filtering banned URLs, extracting text, and LID**) can be easily configured in a **DataTrove** pipeline. It already comes with pretty good default configurations for its filters, and thanks to the work of Penedo et al. (2025), **FineWeb-2 provides us with strong starting points for language-specific filtering across +1000 languages (🧶).**

Regarding LID:

- ✔ FastText → supports 176 languages
- ✔ GlotLID-v3 → **+2000 labels**
- 💡 **Combining them gives you a very nice/strict filter.**
- 💡 **If code switching is something you like to be able to filter/detect, take a look at the LID-tool.**

  >> extendable 🧶

EXERCISE WARM-UP

READ THE DOCS (🔗)

>> pipeline: a list of processing steps to execute
>> executor: runs a specific pipeline
>> job: the execution of a pipeline
>> task: a job is comprised of multiple tasks
>> file(s): .jsonl, .parque, .warc, etc.
>> shard: a group of input data (shards ~ tasks)
>> worker: compute resource (CPU cores)

With 96 workers, you can execute 96 tasks simultaneously (one per CPU core). Once a worker finishes a task, it will start processing the next task in the queue.

QUESTION: Why are we using CPUs and not GPUs?

```python
from datatrove.pipeline.readers import JsonlReader
from datatrove.pipeline.filters import SamplerFilter
from datatrove.pipeline.writers import JsonlWriter
from datatrove.executor import LocalPipelineExecutor


stage1 = LocalPipelineExecutor(
  pipeline = [
    # Read some data
    JsonlReader(data_folder="/my/input/path"),
    # Do something
    SamplerFilter(rate=0.5),
    # Save the result
    JsonlWriter(output_folder="/my/output/path")
  ],
  tasks=16,
  workers=8,
  logging_dir="./logs"
)

# Lunch 🚀
stage1.run()
```

EXERCISE WARM-UP

CLUSTER HYGIENE (🖇️) AND LOGGING (🖇️)

You don’t want/need 1 billion files.

You don’t need a cache of all of your failed jobs.

File sizes should be optimized for your file system.

Key metrics should be easy to inspect.

Automate everything you can (but be safe!).

```
================================================================================
POST-PROCESSING: PT
================================================================================
📂 Found 3 JSONL files
--------------------------------------------------------------------------------
📊 STATISTICS FOR 'PT'
--------------------------------------------------------------------------------
  New Documents Added    :        123,456
  New Tokens Added       :      1,845,678
  ----------------------------------------
  Total Documents        :      1,234,567
  Total Tokens           :     18,345,678
  Avg Tokens/Document    :         400.22
--------------------------------------------------------------------------------
✅ Post-processing for 'pt' completed.

================================================================================
POST-PROCESSING: BN
================================================================================
📂 Found 2 JSONL files
--------------------------------------------------------------------------------
📊 STATISTICS FOR 'BN'
--------------------------------------------------------------------------------
  New Documents Added    :         12,345
  New Tokens Added       :        345,678
  ----------------------------------------
  Total Documents        :        345,678
  Total Tokens           :      3,456,780
  Avg Tokens/Document    :         500.00
--------------------------------------------------------------------------------
✅ Post-processing for 'bn' completed.

================================================================================
FINAL SUMMARY - ALL LANGUAGES
================================================================================

Language        Documents          Tokens          Avg Tokens/Doc
--------------------------------------------------------------------------------
bn                345,678         3,456,780              400.22
pt              1,234,567        18,345,678              500.00
--------------------------------------------------------------------------------
TOTAL           1,548,135        21,925,908              450.11

--------------------------------------------------------------------------------
📈 NEW DATA ADDED IN THIS RUN
--------------------------------------------------------------------------------
  Total New Documents    :      1,548,135
  Total New Tokens       :     21,925,908
================================================================================

✅ All language-specific processing completed.
```

EXERCISE

QUALITY AND DEDUPLICATION

Less Is More

WHAT DEFINES “GOOD” TEXT?

As we dig deeper into building a full dataset curation pipeline, our filtering strategies become more **elaborate** and **specific**. The filters seen before mainly helped us clean **what is obviously undesired** (e.g., SPAM, text not in our languages of interest, etc.). Now, we want to define **a minimum quality threshold** for everything that passed the first filtering stage.

QUESTION: What rules could you come up with for separating ***"text that is good for training LLMs"*** and text that is better left unused?

- ✓ They should have a **minimal length**.
- ✓ They should **not be made of only symbols and numbe**rs.
- ✓ They should have a **"natural" rate of change between symbol types** (e.g., letters and punctuation).
- ✓ They should **not have long repeating strings** of the same character.
- ✓ The entire dataset should **not be (silently) a repetition of just a few documents**.

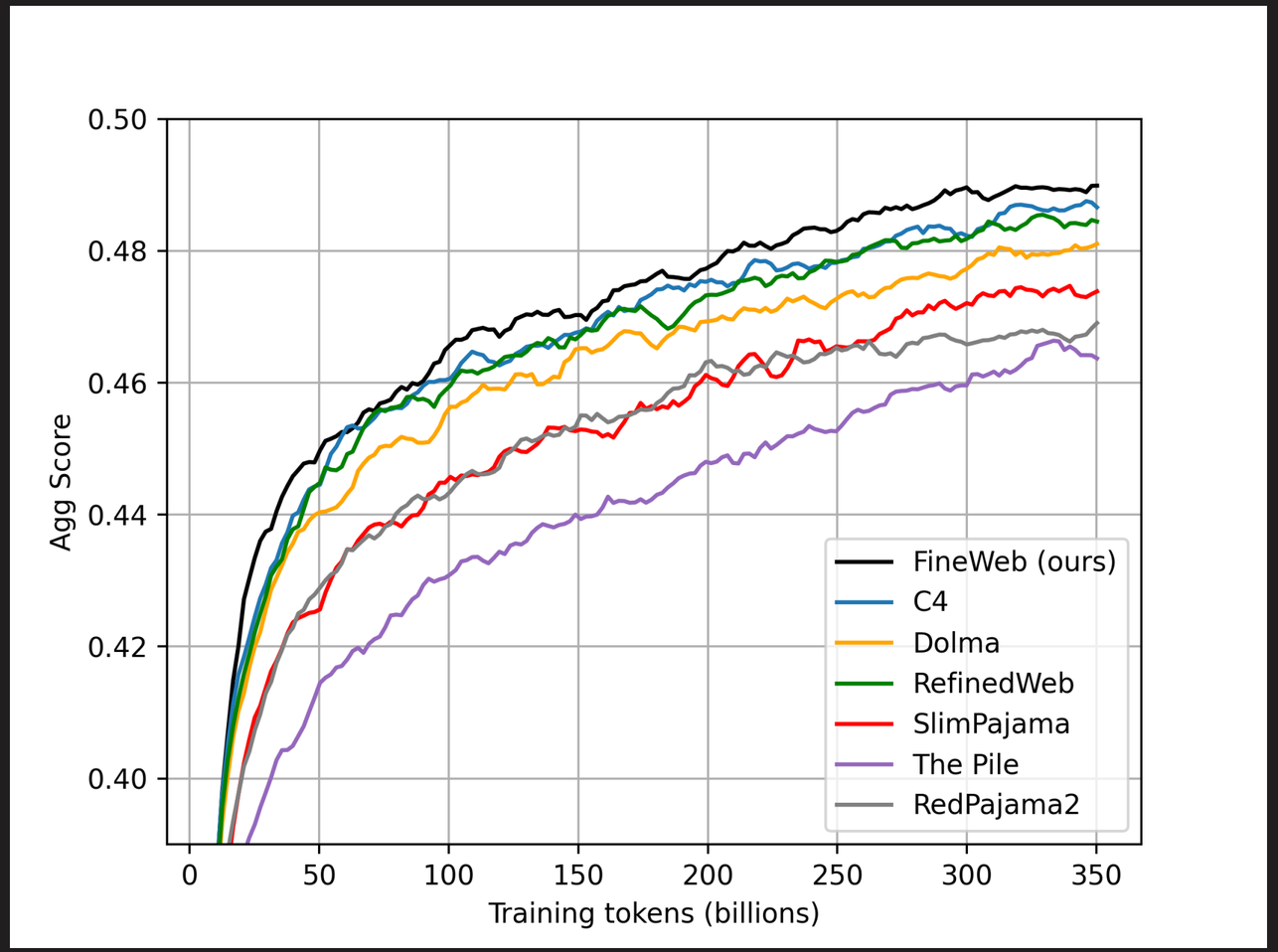
QUALITY FILTERING / HEURISTICS

Heuristic filtering is the process of cleaning a dataset by **removing undesirable or low-quality content**. It typically involves applying a series of **heuristic rules** at both the line and document levels. These are supposed to be **simple and fast to run** (the complete **opposite of an LLM-as-a-filter/judge approach**).

- ✓ **C4** (Colossal Clean Crawled Corpus. 2019). **STILL RELEVANT! Good filters** (📎).
- ✓ **MassiveText** (Scaling Language Models - Gopher 280B. 2021) (📎)
- ✓ **FineWeb (**Part 1 and Part 2, 2024-2025). **Clever methodology for designing filtering heuristics** (📎).

- 💡 Collect **statistics/metrics** (high vs. poor quality).
- 💡 Find which metrics have the greatest **Wasserstein distance**.
- 💡 **Empirically chose a threshold** that would make lower quality closer to higher quality.
- 💡 Validated the resulting filter via **ablations**.

STATE OF THE ART IN QUALITY FILTERING



Source → Hugging Face

QUALITY FILTERING / DEDUPLICATION

Deduplication is the process of **eliminating redundant or highly similar text samples**, which not only **reduces wasted training compute** but also **mitigates** overly repetitive patterns that lead to **memorization** rather than generalization. Models trained on deduplicated datasets tend to produce **more robust language understanding** and **less regurgitation of verbatim web content** (📎).

- ✓ Currently, the standard/battle-tested algorithm for deduplication is MinHash (**ExactHash** fails with near-duplicates, **Sentence-Level** is computationally expensive). It efficiently **approximates the** Jaccard Index (a metric of set similarity) without performing a full set comparison.

- ✓ **MinHash** also **scales efficiently across many CPU nodes** and allows us to **tune similarity thresholds** (by controlling the number of hashes/buckets used) and the length of subsequences considered (by controlling the n-gram size).

QUALITY FILTERING / DEDUPLICATION

Parameter	↑ Increase Effect	↓ Decrease Effect
N-gram size (shingles)	Stricter matching (longer phrases)	Looser matching (catch more)
Hashes	More accurate estimates	Faster, noisier estimates
Buckets	Higher recall (catch more)	Higher precision (fewer false positives)

A CAVEAT ON DEDUPLICATION

- ✓ The belief that **more deduplication inherently improves model performance does not hold to the extreme**. Excessive deduplication, particularly across datasets**, can discard valuable, higher-quality data**.

- ✓ **This should guide you to the following intuition:** most of the performance gains from deduplication come from **removing large-scale duplicates** repeated across all dumps, and ***not from aggressively pruning smaller, less-redundant clusters*** ().

💡 ***Deduplication should enable you to define what should be repeated and when!***

SCALING LAWS FOR DATA CONSTRAINED SETUPS

How do language models scale when training data is **limited/constrained? How does data repetition influence final performance?** This is a major question in low-resource scenarios, for which Muennighoff and collaborators gave a tentative answer. They conducted large-scale experiments varying **data repetition** and **compute budget** (up to 900 billion training tokens), using the C4 and OSCAR datasets with varying numbers of unique tokens.

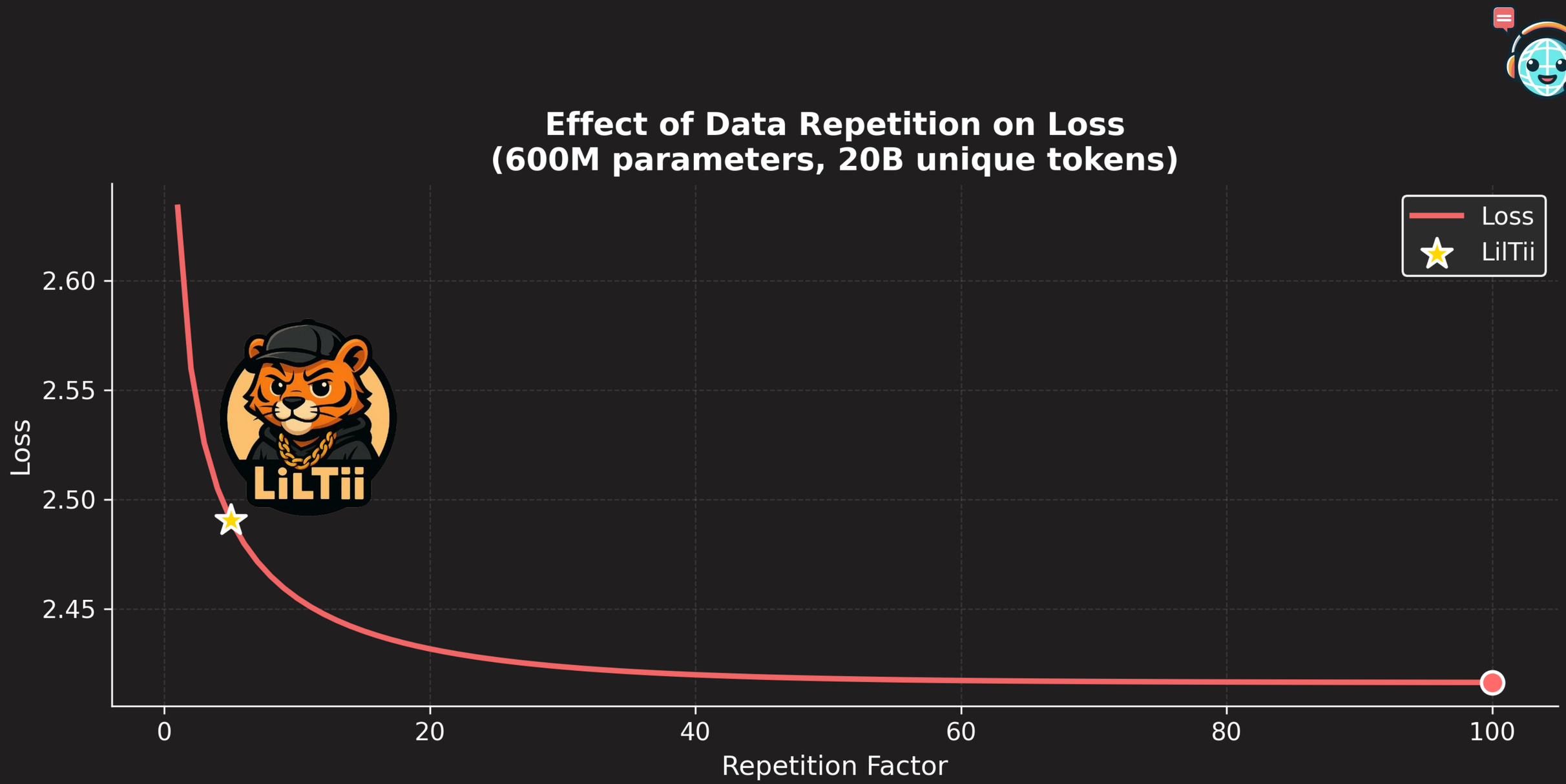
Key Findings:

- ✓ **Data Repetition Trade-Off:** When data is limited, training smaller models for more epochs (with data repetition) **outperforms training larger models for fewer epochs.**
- ✓ **Augmentation and Depud:** Mixing Python code with natural language data while having control over the duplication factor tends to yield better results.
- ✓ **A General Heuristic: T**raining with up to 4 epochs of repeated data yields negligible changes to loss compared to having unique data.

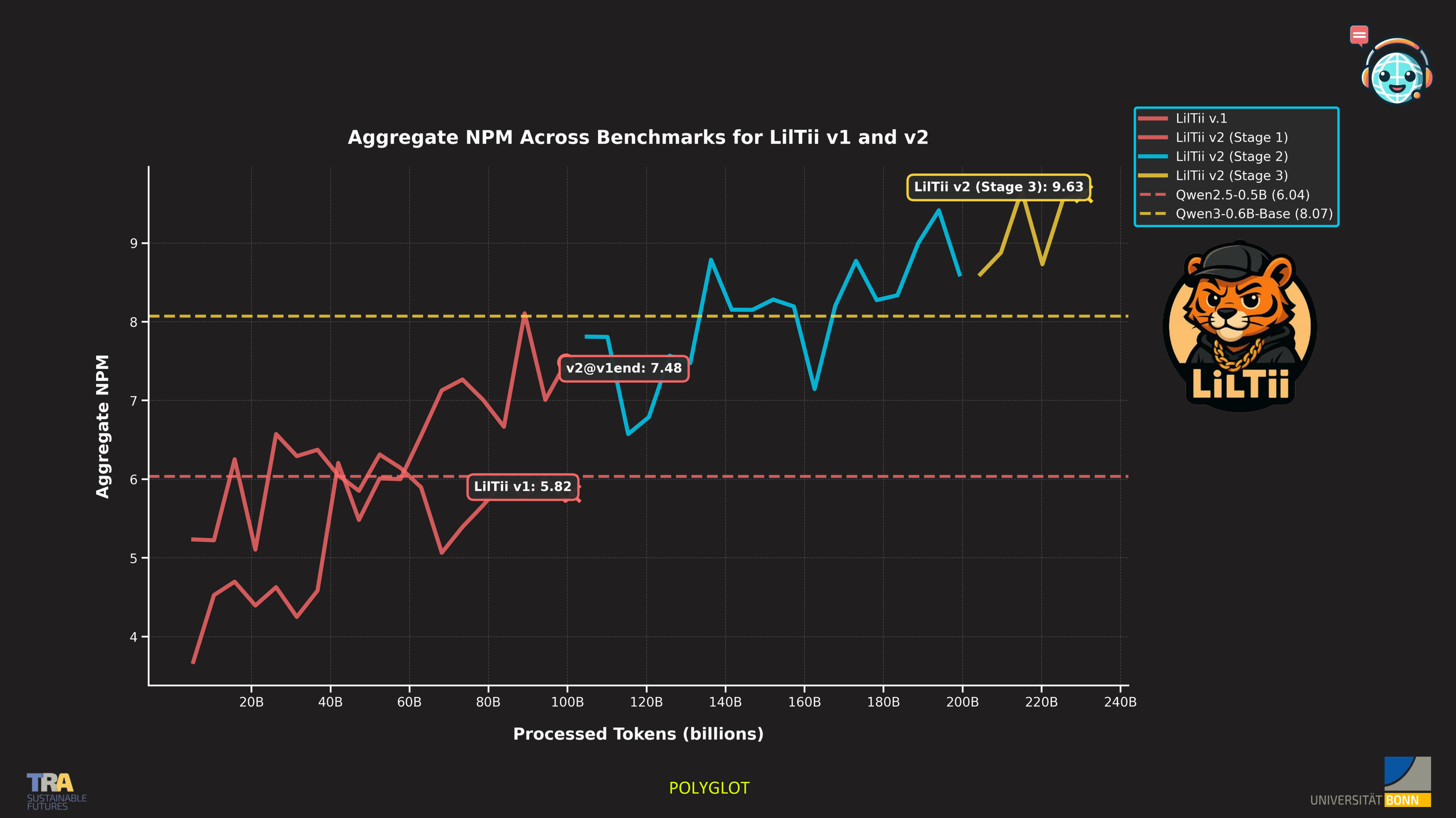
Must read: Scaling Data-Constrained Language Models.

```python
# Parametric Fit Formula from Muennighoff et al. (2023)
# Source: https://arxiv.org/abs/2305.16264
import numpy as np

a, b, e, alpha, beta, rd_star, rn_star = [6.255414, 7.3049974, 0.6254804, 0.3526596, 0.3526596,
15.387756, 5.309743]
A = np.exp(a)
B = np.exp(b)
E = np.exp(e)
G = ((alpha*A)/(beta*B))**(1/(alpha+beta))

def D_to_N(D):
    return (D * G)**(beta/alpha) * G

def scaling_law(N, D, U):
    """
    N: number of parameters
    D: number of total training tokens
    U: number of unique training tokens
    """
    assert U <= D, "Cannot have more unique tokens than total tokens"

    RD = np.maximum((D / U) - 1, 0)
    UN = np.minimum(N, D_to_N(U))
    RN = np.maximum((N / UN ) - 1, 0)

    L = E + A/(UN + UN*rn_star*(1-np.exp(-1*RN/rn_star)))**alpha + B / (U + U * rd_star * (1 -
np.exp(-1*RD/(rd_star))))**beta
    return L

# LilTii example:
print(scaling_law(600e6, 100e9, 20e9))
# >> 2.4905030254657876
print(scaling_law(600e6, 140e9, 20e9))
# >> 2.4715302066783953 < -- Still okay to repeat!
```

Scaling Data-Constrained Language Models


Effect of Data Repetition on Loss
(600M parameters, 20B unique tokens)
Loss
LilTii
2.60
2.55
2.50
2.45
Loss
0
20
40
60
80
100
Repetition Factor
LiLTii

POLYGLOT


Aggregate NPM Across Benchmarks for LilTii v1 and v2
LilTii v.1
LilTii v2 (Stage 1)
LilTii v2 (Stage 2)
LilTii v2 (Stage 3)
Qwen2.5-0.5B (6.04)
Qwen3-0.6B-Base (8.07)
LilTii v2 (Stage 3): 9.63
v2@v1end: 7.48
LilTii v1: 5.82
Aggregate NPM
Processed Tokens (billions)
4
5
6
7
8
9
20B
40B
60B
80B
100B
120B
140B
160B
180B
200B
220B
240B
LiLTii

POLYGLOT

QUALITY AND DEDUPLICATION

"Less Is More". Even in data-constrained scenarios, we still have (**up to a certain limit**) heuristics and rules, dictated by scaling laws (**and confirmed by thousands of GPU hours of experimentation**), that help us **allocate our FLOPs most effectively**.

- ✔ Implement quality with language-specific parameters.
- ✔ Build a 4-stage MinHash deduplication pipeline using LSH bucketing.
- ✔ Apply text formatters (e.g., PII removal) to improve the final output (text extraction fixes!).
  - ✔ **Bonus Challenge 1:** Implement FineWeb quality filters from scratch
  - ✔ **Bonus Challenge 2:** Implement MinHash deduplication (the dummy version)

EXERCISE

LLM-AS-A-FILTER AND SYNTHETIC DATA GENERATION

Bootstrapping from Multilingual Foundations

WHAT DEFINES “GOOD” TEXT, PART-II

“Good” text cannot be defined solely by surface features such as punctuation rates or letter frequencies. **An educational essay on the Hundred Years’ War can possess the same level of textual “goodness” as a grammatically correct essay advocating white supremacy**. When we want more **aesthetic or value-based definitions** of quality, simple heuristics often fail to capture what we actually mean. For instance, the **presence of a word like *hypothesis* does not necessarily place a text within the scientific domain**, nor does it guarantee that the text represents good science.

From this realization, and the fact that LLMs can be used quite robustly for text classification, comes the idea of an LLM-as-a-filter (or Judge).

>> Gunasekar, et al. (2023). Textbooks Are All You Need. **ChatGPT 3.5 → Phi-1** (🔗)

>> Grattafiori et al. (2024). The Llama 3 Herd of Models. **Llama 2 → Llama 3** (🔗)

>> Penedo et al. (204). The FineWeb Datasets. **Llama2-70B-Instruct → Many models (🔗)**

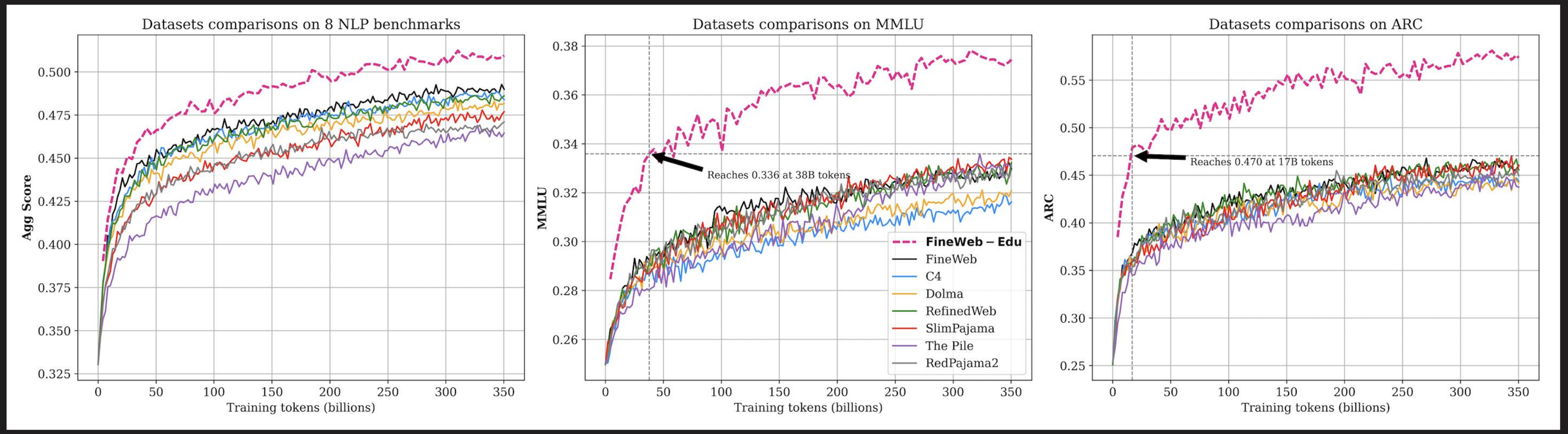
THE CASE FOR THE EDUCATIONAL FILTER

In the case of **FineWeb-Edu**, the authors used **LLaMA-3-70B-Instruct** to generate annotations assessing educational value on a Likert scale (LLMs tend to perform better with Likert-style judgments than when asked to produce a single bounded score). Using these annotations, they trained a lightweight classifier to label a portion of the FineWeb corpus. Samples scoring above a predefined threshold were then selected to form the **FineWeb-Edu** dataset.

- ✔ The **prompt** (📎)
- ✔ The **base** model (📎)
- ✔ The FineWeb-Edu **classifier (📎)**

FineWeb-Edu shows that with this kind of filtering, you can achieve good performance with significantly less data (e.g., **you need 10x fewer tokens** than C4 and Dolma to **match MMLU results** for a 1.8 billion-parameter model).

THE CASE FOR THE EDUCATIONAL FILTER

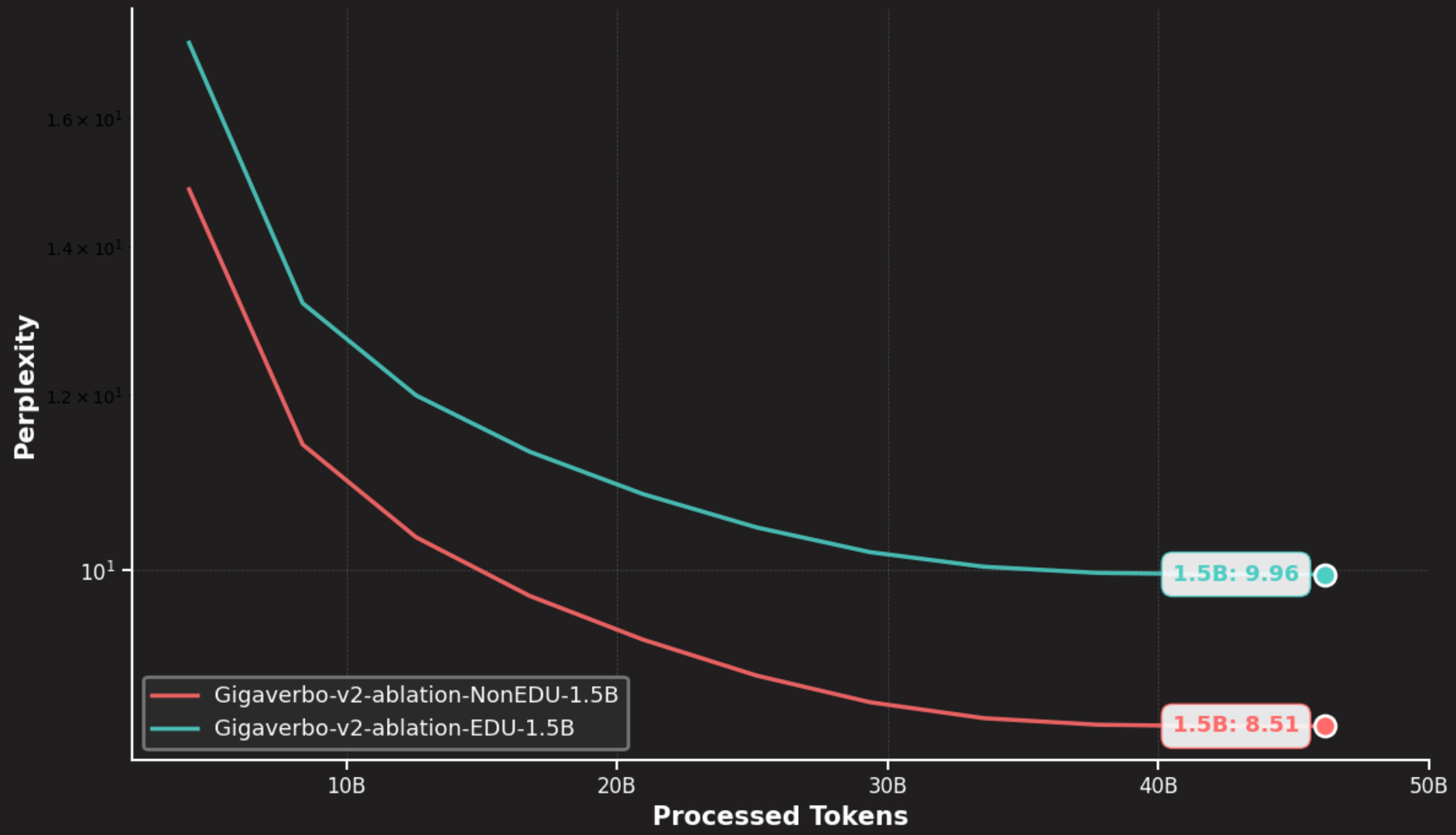


Source → Hugging Face

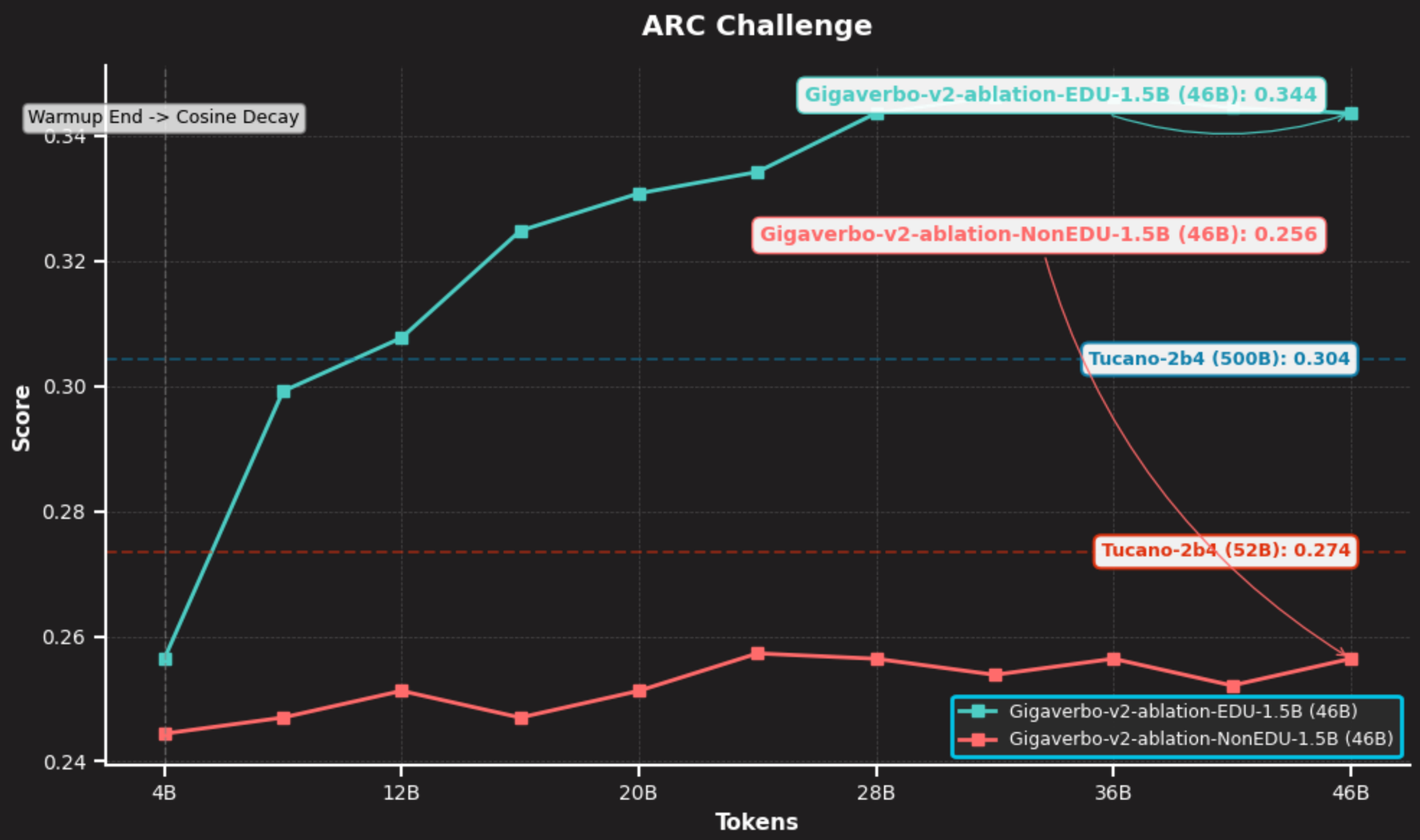
REPRODUCING AND EXTENDING

According to our experiments, models such as **Qwen2.5** and **Qwen3** offer a strong alternative to LLaMA-3-70B-Instruct for low-resource languages. Although they may fall short in terms of fluent text generation, they perform very well as classifiers—at least for the languages we evaluated, including **Bengali**, which is particularly low-resource. In addition to their strong performance, these models are also (mostly) free to use under the **Apache 2.0** license.

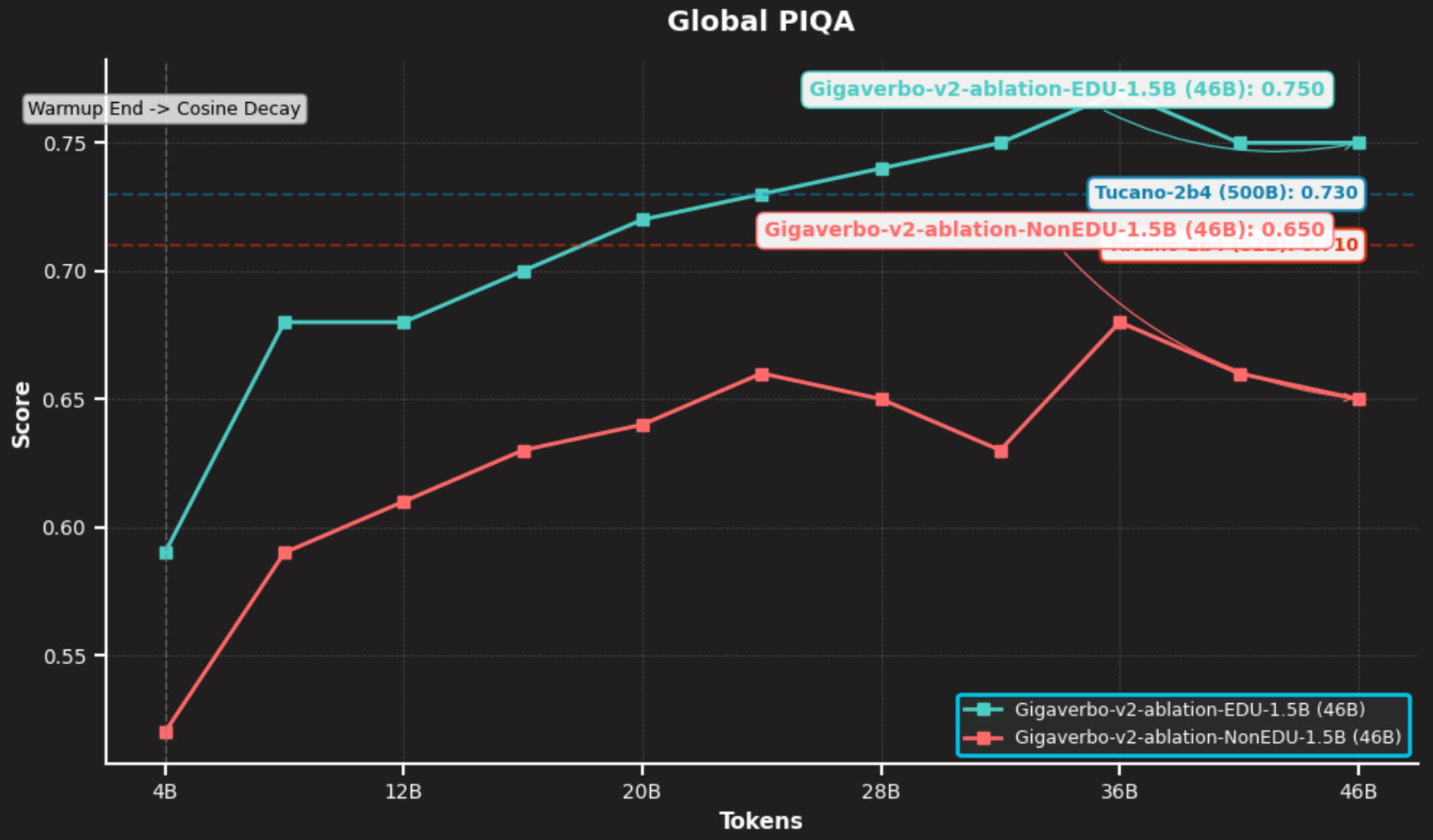
Moreover, this methodology can be extended to other areas of interest. In our case, we applied it to dataset detoxification; manual inspection showed it to be highly effective at removing **NSFW** content.



QUESTION: ***Why is the Non-EDU model achieving a lower loss? Why is it worse? Did the people at HF lie to us?!***



What we really care about is benchmark performance.



What we really care about is benchmarks that target our domain (e.g., language) of interest.

FROM JUDGES TO GENERATORS

Once you have a **working LLM-as-a-filter pipeline**, adapting it to generate new text (rather than merely classifying existing text) requires only **minimal modifications**. For example, instead of scoring inputs, the model can be prompted to **expand or transform a seed text** such as a Wikipedia article. With sufficient compute, this shift opens the door to **generating synthetic data**. Synthetic data provides a powerful way to improve language models in domains that are underrepresented or rare in a given language.

Cosmopedia (🗞️): Mixtral-8x7B-Instruct-v0.1 **(**🛠️ llm-swarm**) → 25 billion tokens.**

- 🛠️ synthetic-data-kit **(**🗞️**)**
- 🛠️ DatTrove **(**🗞️**)**
- 🛠️ All of these use vLLM under the hood, the ***fastest inference engine*** in the business **(**🗞️**).**
  - 💡 Smart KV cache management makes LLMs go brrrrrr.

Investing in synthetic data generation can be far **more impactful** (in terms of long-term contributions to the community) than training any single model. Models come and go, but datasets (and benchmarks) tend to ***persist*** **and continue shaping research for much longer.**

LESSONS FROM COSMOPEDIA

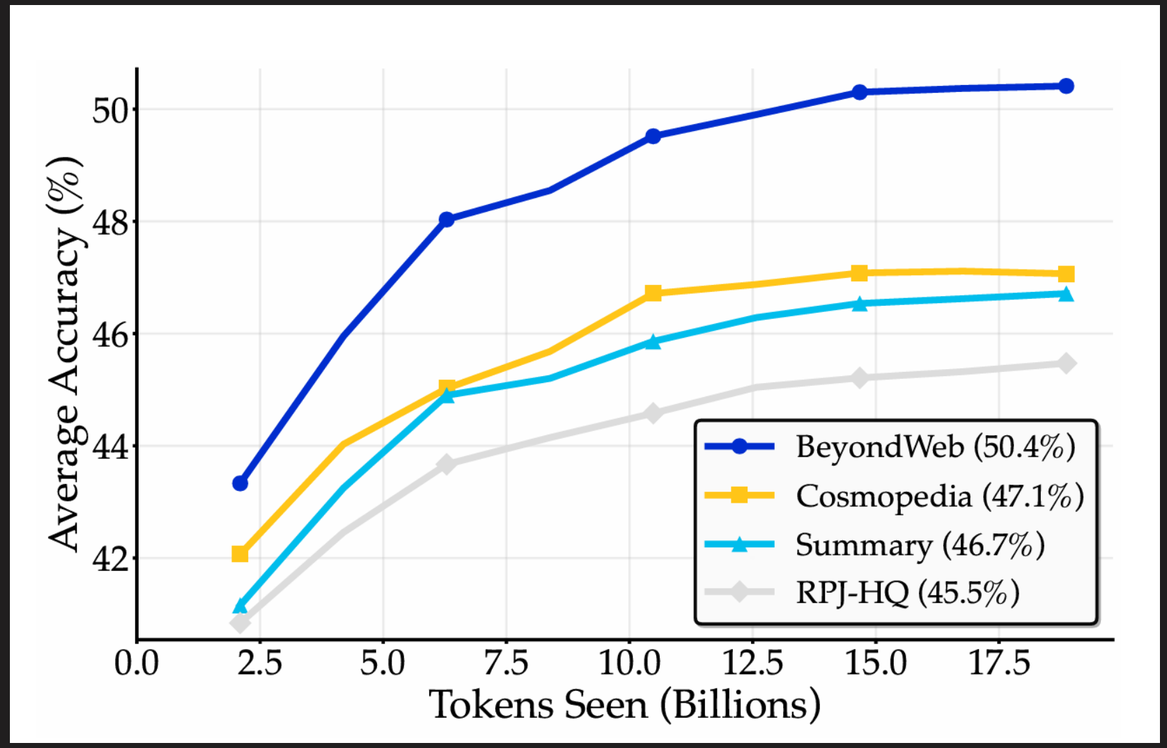
1. Effective prompt engineering accounts for roughly **80% of the work**.
2. Prompts can be **derived automatically from data**, rather than written entirely by hand (scalable!).
3. Using **pre-existing text as a seed** (combined with a prompt) helps with diversity (the Phi-1.5 trick).
4. Prioritize **high-quality seed sources**, such as **Wikipedia**, to anchor generation.
5. Introducing **audiences** or **personas** further increases output diversity.
   - ✔ “Write for an academic audience …”
   - ✔ “You are a PhD-level expert in …”

LESSONS FROM BEYONDWEB

BeyondWeb **(Maini et al. 2025) is another synthetic data generation framework that produces high-quality synthetic data for pretraining, operating on principles distinct from those of Cosmopedia.**

💡 **Rephrased** low-quality text is (**almost**) **as good as high-quality text**.

💡 **Increasing per-token information** density is one mechanism (among potentially many) by which synthetic data improves pre-training.

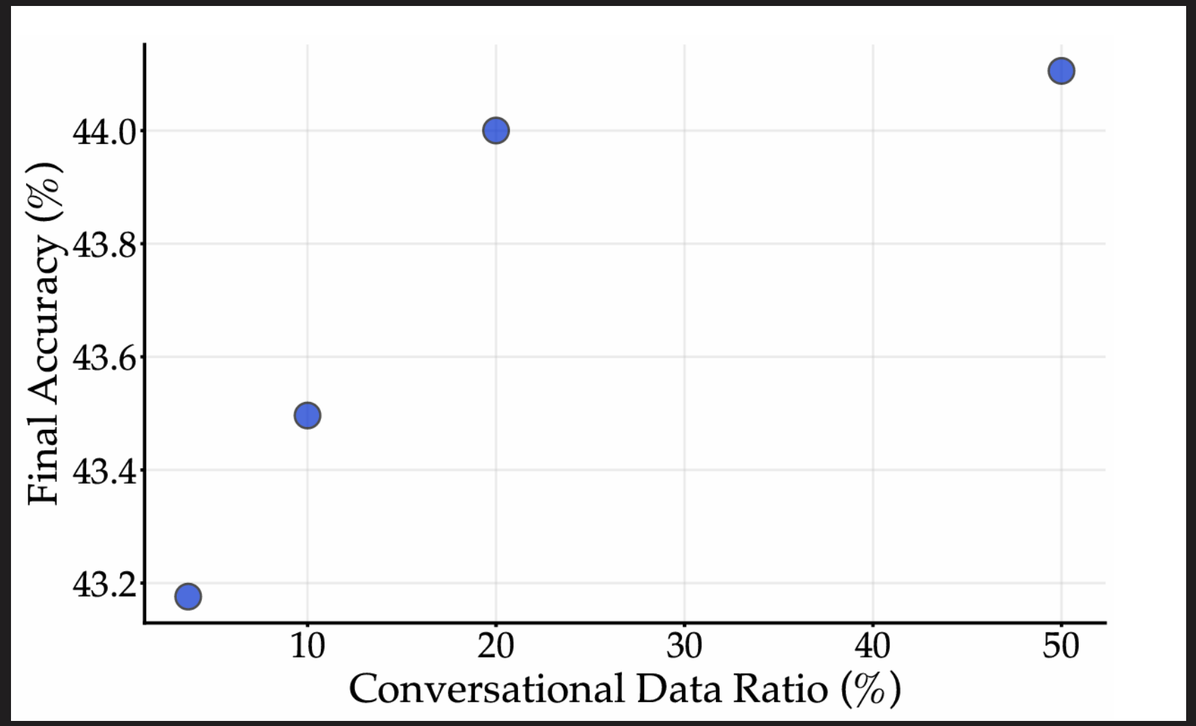


BeyondWeb: Lessons from Scaling Synthetic Data for Trillion-scale Pretraining

LESSONS FROM BEYONDWEB

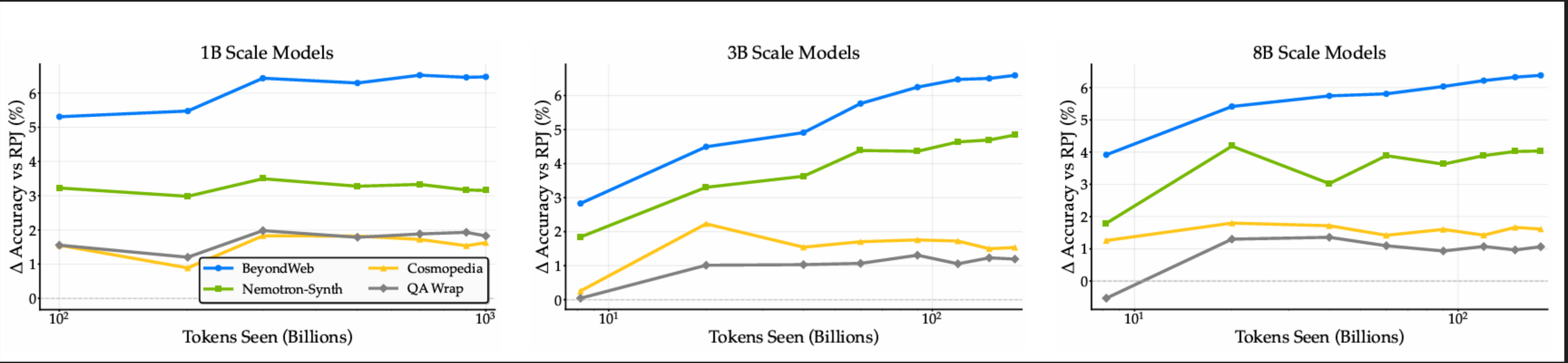
Given limited high-quality data, it is **more beneficial to use it as seed data for synthetic rephrasing rather than low-quality data**, despite the possible drawback of repeating knowledge (**deduplication is not all you need**).

Using natural **conversational data improves downstream task performance**, though these improvements are modest and exhibit diminishing returns as the proportion of conversational data increases.



BeyondWeb: Lessons from Scaling Synthetic Data for Trillion-scale Pretraining

LESSONS FROM BEYONDWEB



BeyondWeb: Lessons from Scaling Synthetic Data for Trillion-scale Pretraining

Models **adapt fast to consistent formats**. Synthetic data quality **improves when we use different families of LLMs** (e.g., Llama, Qwen, Mistral).

The **quality of synthetic data increases** as generator size increases **from 1B to 3B**, but **saturates at 8B.**

GIGAVERBO-SYNTH

A little on our methodology for developing **GigaVerbo-v2-Synth (🍥)**.

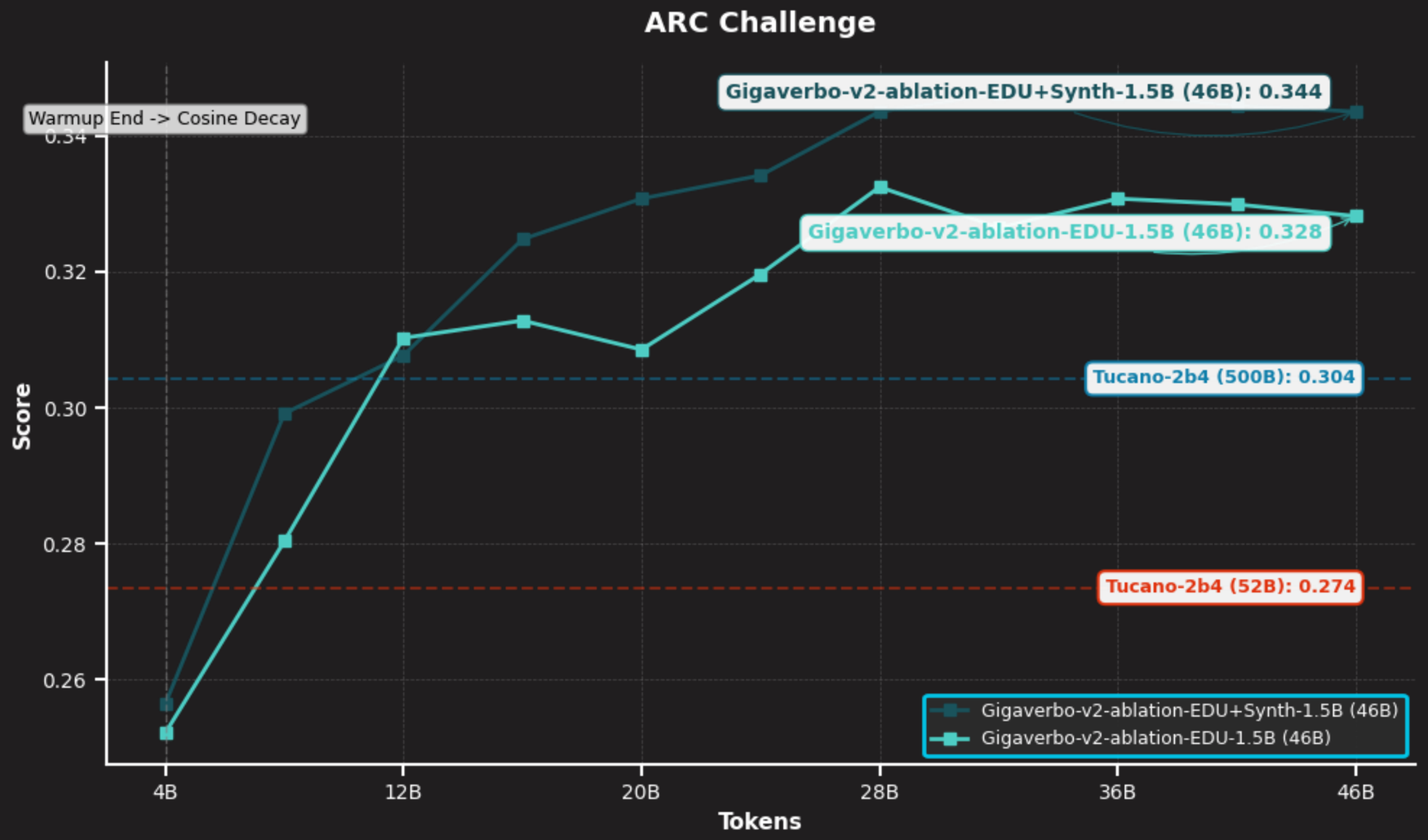
- ✔ For our first run, we stuck with the Qwen2.5 and Qwen3 families (**wide multilingual support**)
- ✔ **For generating pre-training data**, we mostly used **7B-8B** models.
- ✔ For **Math/Code/SFT/DPO** style data, we also used more costly generators (**14B-32B**).
- ✔ A **quantized 32B** can give you a nice **trade-off between speed and capability**.
- ✔ Using **English sources** as seeds for **translate-rewrite/summarize** worked really well.

If you don't have compute resources to run your own pipeline, you can use models served via APIs. Then the bottleneck becomes how much you can pay for API calls. Currently, you need **~USD $1,500 to generate 1 billion tokens with ChatGPT 3.5 Turbo, and ~USD $200 with Qwen-Turbo.**

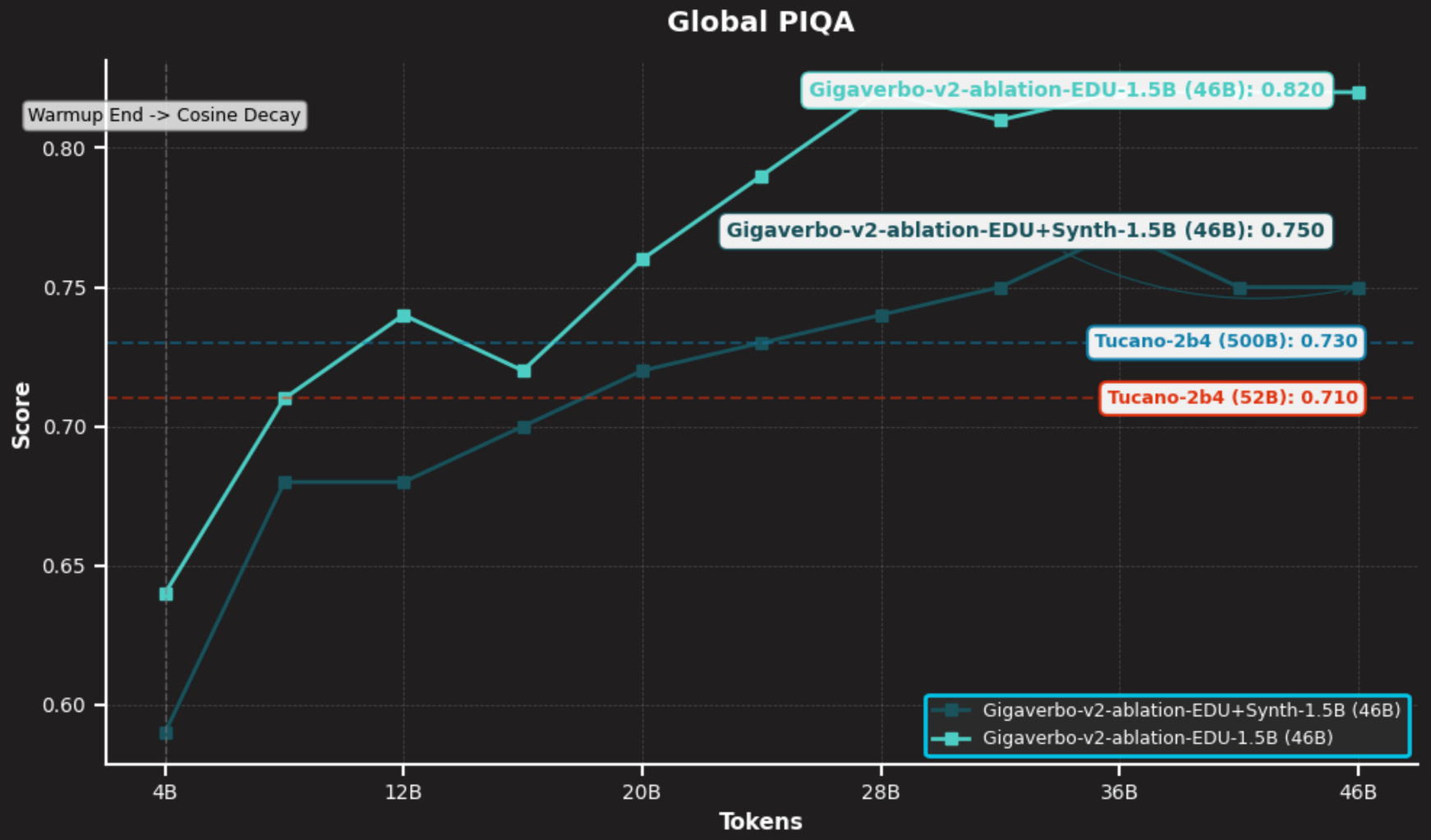
GIGAVERBO-SYNTH

A little more on the details:

- ✔ Generated during a **cumulative period of 4 months**.
- ✔ Custom vLLM setup using **16 x NVIDIA A40 (MLGPU partition)**.
- ✔ We estimate that we consumed around **~46,000 GPU hours**.
- ✔ This amounts to approximately **14,400 kWh of energy** consumption.
- ✔ Which amounts to approximately **5.4 metric tons of CO2 equivalent**.
- ✔ **In total: 11,237,546 samples. 26 GB of text (10 billion tokens).**



The mix of "organic" educational content + the synthetic counterpart outperforms any solo baseline …



The mix of "organic" educational content + the synthetic counterpart outperforms any solo baseline …
For some benchmarks*

POLYGLOT

LEARNED FILTERS AND SYNTHETIC GENERATION

LLMs are powerful tools for building other LLMs. Learning how to design efficient **quality-control pipelines** or **synthetic generation swarms** may be the only **scalable** way to obtain high-quality tokens **in some low-resource languages**.

- ✔ Learn how to set up and run LLM inference (via vLLM) on an HPC cluster.
- ✔ Learn how to create a simple pipeline that uses LLMs for text classification or generation.
- ✔ Run a small-scale experiment to evaluate the effectiveness of the approach.

EXERCISE

DECONTAMINATION, RECIPES, AND MIXTURES

Bonus Content

SOME BONUS LORE

It is not possible for us to **cover all the important steps involved in creating and curating data for LLMs**. There are simply **too many** of them, and too **many factors to consider**.

Moreover, I hope none of you is currently under the **impression that any of this is simple and straightforward**. The road is **extremely non-linear**, and more than once we need to go back to square 0 and **rethink**, **re-try**, **re-run everything**. **Be pessimistic when estimating your resources.**

Now, let me very quickly fast-track a few topics that I would *strongly* recommend you explore further through your own research and self-learning. **These were things that greatly helped us develop our own models.**

DECONTAMINATION

Benchmarks are the candles that guide us in the dark. Few things will derail a project faster (or at greater cost) than **optimizing for a metric that does not measure what you think it does**.

We will devote some time tomorrow to talking about benchmarks, so I will leave some details aside for now. The key point I want to emphasize is that **decontaminating your dataset from leaked benchmarks is a priority.** You should **assume that most benchmarks released prior to the current year have already leaked into the broader data ecosystem**.

Good evaluations possess specific properties (more on this later), and once your training data is contaminated with the evaluations you rely on, the results of those evaluations **cease to be meaningful**.

DECONTAMINATION VIA K-"TOKEN" MATCHING

A simple method for decontamination: ***contiguous k-token (n-gram) matching*** (🧶).

- ✓ Most contamination approaches rely on ***n-gram matching***, where text from evaluation datasets (e.g., prompts or reference answers) is compared against the data being filtered (Muennighoff et al. 2025). However, operating at the **word level** can be fragile, as it is **sensitive to encoding differences**, **diacritics**, **tokenization choices**, and **other orthographic variations**.

- ✓ To mitigate these issues, we adopt a **token-based approach**, performing **contiguous k-token matching** rather than word-level n-gram matching. By operating directly on the model's tokenized representation, this method provides greater robustness across languages and writing systems, particularly in multilingual and low-resource settings.

DATA MIXTURES AND RECIPES

Gathering all your well-filtered text into a single dataset and training for a few epochs with a cosine-decay learning rate scheduler is about as **vanilla as it gets**. It works and serves as a reasonable baseline, but it is far from optimal.

- ✓ Modern training runs are typically **multi-stage** and rely on a deliberately designed (and often ablated) **curriculum**, in which the data mixture evolves over the course of training.

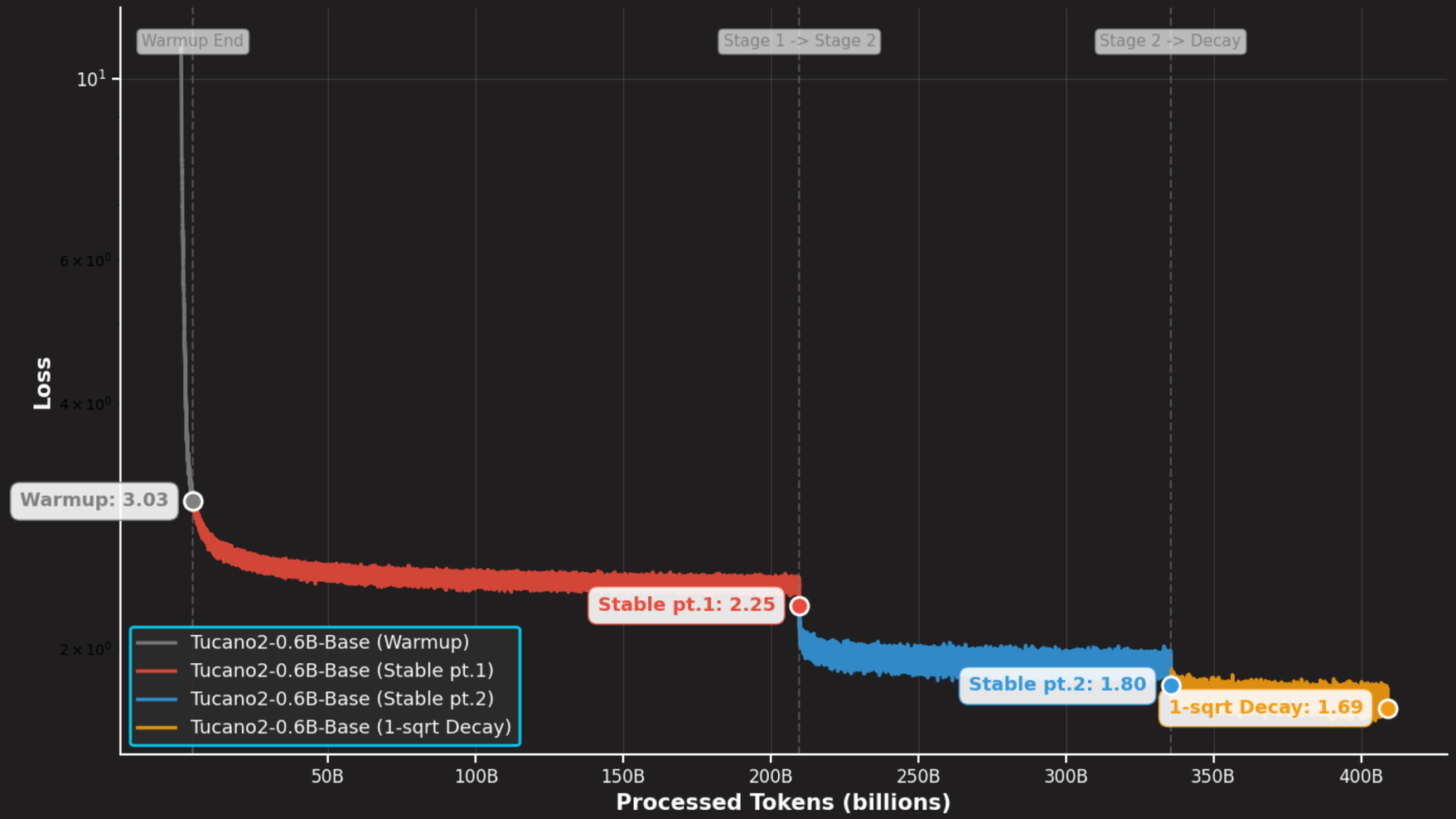
- ✓ A language **model's final behavior is strongly influenced by the data it encounters toward the end of training**. This observation motivates a practical strategy: **emphasize larger, more abundant data sources early on and progressively mix in smaller, higher-quality datasets later.**

DATA MIXTURES AND RECIPES

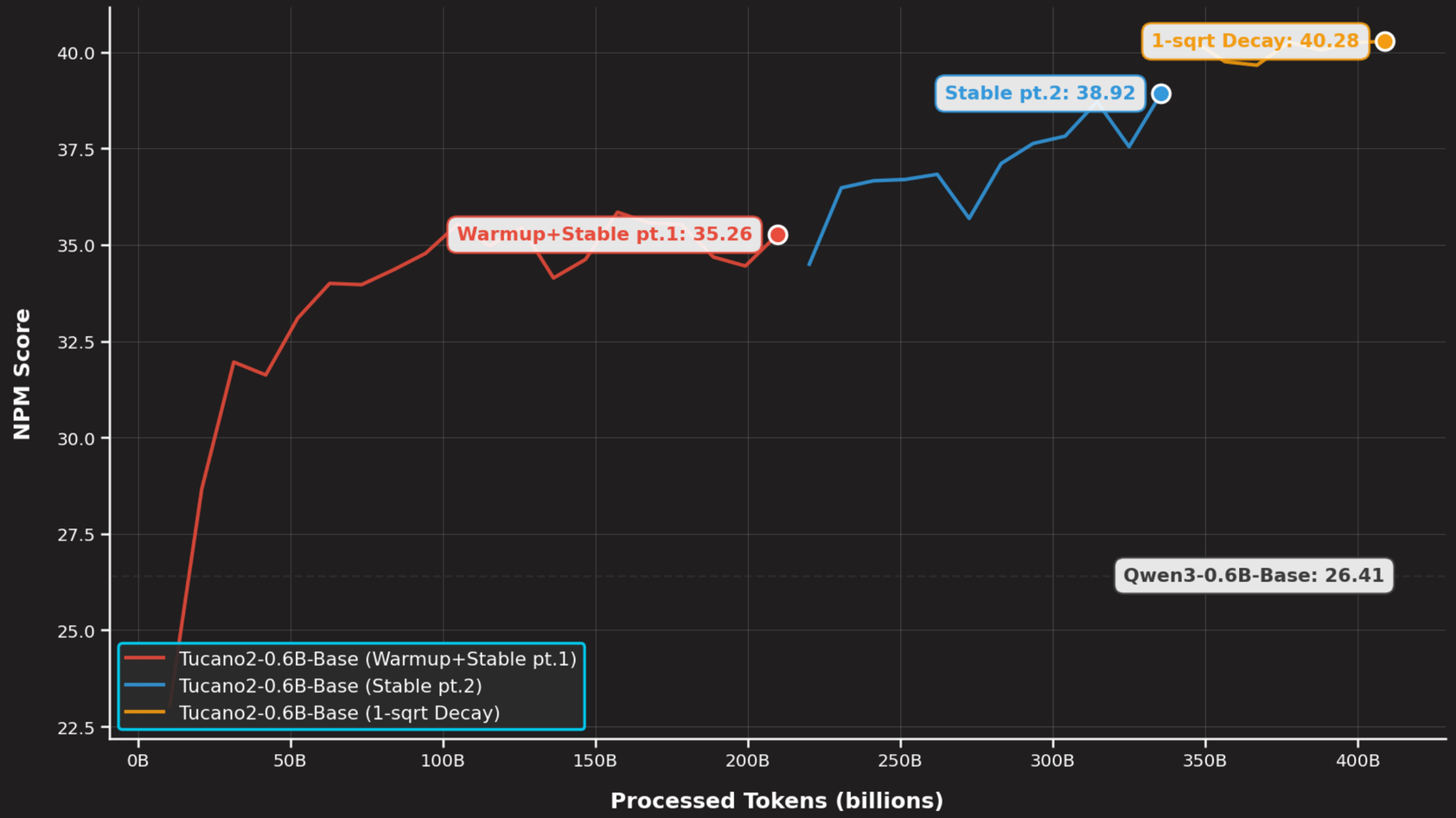
There are many **recipes already battle-tested** that we can use to create our own.

- ✔ **2 OLMo 2 Furious (📎)**
- ✔ **SmolLM2: When Smol Goes Big (📎)**
- ✔ **SmolLM3: smol, multilingual, long-context reasoner (📎)**
- ✔ **Apertus: Open and Compliant LLMs for Global Language Environments (📎)**

💡 Failure to exactly reproduce results is **normal**, especially when **data mixtures and model scales differ significantly**. There is no escaping the **ablation marathon** if you want to do your best with the resources you have.



Monitor evaluation metrics and adapt dataset mixtures to address specific capability gaps.



Small high-quality data sources are most impactful when introduced during the annealing phase.

BREAK → TOKENIZATION